Editron Cut Engine

Project Status • Architecture • Target Behavior • Roadmap

Purpose: A technical reference for what Editron Cut Engine does today, what was learned during testing, the intended editing intelligence, and the architecture needed to reach the target.

1. Executive Summary

Editron Cut Engine is being designed as a speech-aware automatic video editing engine for talking-head and short-form videos. The goal is not merely to remove silence. The target is to reconstruct the cleanest, most natural version of the speaker's intended dialogue while preserving every meaningful point.

The current prototype performs speech/silence analysis, uses Faster-Whisper for transcription and word-level timing, detects repetition candidates, creates an edit plan, and renders the resulting video with FFmpeg. Testing shows that silence removal is already working well. The main gap is higher-level speech intelligence: fillers, repeated words and phrases, false starts, abandoned takes, and semantic/content preservation.

2. What We Have Built So Far

Component	Role
CLI	Command-line interface for input/output and editing parameters.
VAD	Speech/activity detection for identifying speech and non-speech regions.
Faster-Whisper	Speech recognition and word-level timing; large-v3 is the quality baseline and CPU testing uses int8.
Repetition engine	RapidFuzz-based initial repeated word/phrase detector; current tests pass.
Edit plan / EDL	Separates analysis from rendering and represents keep/cut ranges.
FFmpeg	Executes the approved media cuts.
Preview JSON	Exports edit decisions for inspection and debugging.

3. What We Took From Auto-Editor

Auto-Editor was the conceptual starting point for media activity analysis and timeline-based editing. Its useful foundation is the idea of analyzing audio activity, producing active/inactive ranges, and executing those ranges through a media-processing pipeline.

Editron is intentionally going beyond that model. Activity detection answers approximately *where is audio active?* Editron needs to answer *what did the speaker mean, what was a mistake or bad take, and what can be safely removed without changing the intended message?*

```
Audio activity → speech detection → transcription → word timing → linguistic analysis →
contextual/semantic validation → confidence-based edit decisions → EDL → FFmpeg.
```

4. Current Architecture (v0.1 Foundation)

```
RAW VIDEO → Audio extraction → VAD / speech detection → Faster-Whisper large-v3 + word timestamps →
repetition analysis → edit plan / EDL → FFmpeg → FINAL VIDEO
```

5. Current Testing Status

Area	Status	Observation
Python environment	Working	Dependencies were repaired inside the project virtual environment.
Pytest	Passing	2 tests passed after installing missing RapidFuzz.
CLI	Working	CLI help loads correctly.
Faster-Whisper	Working	large-v3 runs on CPU with int8.
VAD / silence cutting	Working well	Real-video testing shows accurate silent-region removal and speech preservation.
Repetition removal	Initial	Works, but improvement is smaller than the desired target.
Higher speech intelligence	Not yet	Fillers, false starts, bad takes and semantic protection need a dedicated layer.

6. The Real Editron Target

Core target: Editron should not simply remove silence. It should understand the speaker's delivery and reconstruct the cleanest natural version of what they intended to say.

For raw delivery containing hesitation, repetition and restarts, the desired output is a clean version that removes obvious disfluencies and bad starts while retaining the actual information and natural delivery.

7. Speech Problems Editron Must Understand

Problem	Meaning
Silence	Long or unnecessary pauses between speech.
Fillers / disfluencies	um, umm, uh, hmm, er, ah and similar hesitation sounds.
Repeated words	Accidental repetitions such as “I think, I think this is important.”
Repeated phrases	A phrase repeated because the speaker restarted or stumbled.
False starts	A sentence begins, is abandoned, and is restarted in a cleaner form.
Abandoned fragments	Incomplete wording that should not survive into the final edit.
Bad takes / restarts	An earlier incomplete/flawed delivery followed by a better version.
Intentional emphasis	Repetitions such as “very, very important” that should not be removed automatically.
Content loss risk	Any cut that removes a meaningful fact, claim, example, or key point.

8. Critical Design Rule: Candidate Cuts, Not Instant Cuts

The engine should not immediately delete anything merely because one detector flags it. Each detector should produce a candidate edit with evidence and confidence.

```
Candidate → reason → timestamps → transcript context → audio context → confidence →
semantic/content check → allow/reject → final EDL.
```

This safety mechanism prevents aggressive editing from turning human speech into robotic speech.

9. Proposed v0.2 Speech Intelligence Architecture

```
RAW VIDEO
↓
Audio extraction
↓
■■■
```

```
CONTEXT ANALYZER
↓
SEMANTIC VALIDATION
↓
CONTENT-SAFETY CHECK
↓
CONFIDENCE SCORE
↓
FINAL EDIT PLAN
↓
FFmpeg
↓
FINAL VIDEO
```

10. Components and Responsibilities

Component	Responsibility
VAD	Provide robust speech boundaries.
ASR / Whisper	Determine what was said and when, with word-level timestamps.
Filler detector	Find hesitation sounds using transcript + timing + audio evidence.
Repetition engine	Detect repeated words/phrases while distinguishing accidental repetition from emphasis.
False-start engine	Identify abandoned clauses and restarted delivery.
Take/segment analyzer	Compare nearby candidate takes and identify incomplete vs cleaner versions.
Context analyzer	Use neighboring words/sentences to understand whether a cut makes sense.
Semantic validator	Check whether a proposed edit preserves intended meaning.
Content-safety layer	Prevent removal of important facts, names, numbers, claims, instructions, or required dialogue.
Confidence system	Fuse evidence into conservative, configurable cut confidence.
EDL / timeline engine	Turn approved decisions into keep/cut ranges with padding.
FFmpeg renderer	Execute the approved edit plan.

11. Content Preservation — Non-Negotiable

Because the target is to never miss script dialogue or important points, Editron needs a content-preservation gate before rendering.

```
Original transcript → proposed final transcript → compare meaning/content → if important information is lost, reject the cut.
```

Protected content includes medical/technical facts, names, numbers, instructions, claims, examples, and any required script point. A questionable section should be left untouched rather than risking meaningful content loss.

12. Confidence-Based Editing

Situation	Expected behavior
Obvious filler “umm” isolated from meaningful speech	High-confidence candidate.
Accidental “I I” repetition	High-confidence candidate.
“very very important”	Low confidence; preserve unless additional evidence indicates a mistake.
Possible false start	Candidate; require context/semantic validation.
Possible bad take	Require comparison with surrounding take and content preservation.

13. Efficient AI Strategy

Do not send the entire video through a large reasoning model. Use a cascade: cheap/local detectors handle obvious cases; only suspicious regions go to a higher-level reasoning layer.

```
ALL SPEECH → cheap detectors → obvious KEEP/CUT + suspicious candidates → contextual/semantic reasoning → final decision.
```

This reduces cost and latency while reserving stronger reasoning for difficult cases such as sentence restarts, ambiguous repetitions, and bad-take selection.

14. Caption Engine Integration

The long-term architecture should avoid transcribing the same video separately for captions and cutting.

```
VIDEO → MASTER TRANSCRIBER → MASTER TRANSCRIPT + WORD TIMINGS → Caption Engine + Cut Engine + future content engines.
```

This makes transcription a shared platform service and lets multiple Editron features consume the same authoritative word/timestamp data.

15. Editing Modes

Mode	Behavior
Natural	Preserve natural pauses, breathing and emphasis; remove only high-confidence unnecessary material.
Tight	Remove fillers, obvious repetitions and more unnecessary pauses while preserving natural delivery.
Aggressive	Optimize strongly for short-form pacing, but retain the content-safety gate and never knowingly remove important dialogue.

16. Development Roadmap

Phase	Status	Goal
Phase 0 — Foundation	Complete	CLI, VAD, Whisper word timing, initial repetition detector, EDL, FFmpeg rendering, preview JSON.
Phase 1 — Disfluency	Next	Dedicated filler/hesitation detection using audio + transcript evidence.
Phase 2 — Repetition	Next	Word repetition + phrase repetition + intentional-emphasis protection.
Phase 3 — False Starts	Next	Detect abandoned clauses, sentence restarts, and clean replacement boundaries.
Phase 4 — Bad Takes	Planned	Compare nearby takes/segments and select the cleaner complete delivery.
Phase 5 — Semantic Safety	Planned	Validate that edits preserve meaning and important content.
Phase 6 — Confidence Planner	Planned	Fuse all evidence into conservative, configurable decisions.
Phase 7 — Shared Transcript	Planned	Connect Caption Engine and Cut Engine to one master transcription service.
Phase 8 — Production GPU	Planned	Use NVIDIA GPU workers for fast large-v3 processing while keeping CPU local-preview mode.

17. How We Should Measure Success

Metric	Question
Cut precision	When Editron cuts something, was it actually safe to cut?
Cut recall	How many real fillers, repetitions and false starts did it find?
Content preservation	Did any meaningful dialogue or key point disappear?
Naturalness	Does the output still sound like a human naturally speaking?
Runtime compression	How much unnecessary duration was removed?
Processing speed	How long does analysis/rendering take on CPU vs production GPU?

18. One-Sentence Engineering Requirement

Build a speech-aware automatic video editing engine that reconstructs the cleanest natural version of the speaker's intended dialogue by detecting silence, fillers, repetitions, false starts and bad takes, while using contextual/semantic validation to preserve every meaningful script point.

19. Team Mental Model

INTELLIGENCE — VAD, ASR, disfluency detection, repetition, false-start analysis, contextual/semantic reasoning.
DECISION — candidate cuts, confidence, content-safety checks, final EDL.
EXECUTION — FFmpeg renders the approved timeline.

The intelligence can evolve repeatedly without rewriting the media renderer. This separation is a core architectural principle for Editron.

Document note: This document records the Editron Cut Engine work and target direction established in the current project discussion. It is intended as a development reference before implementing the next intelligence layer.